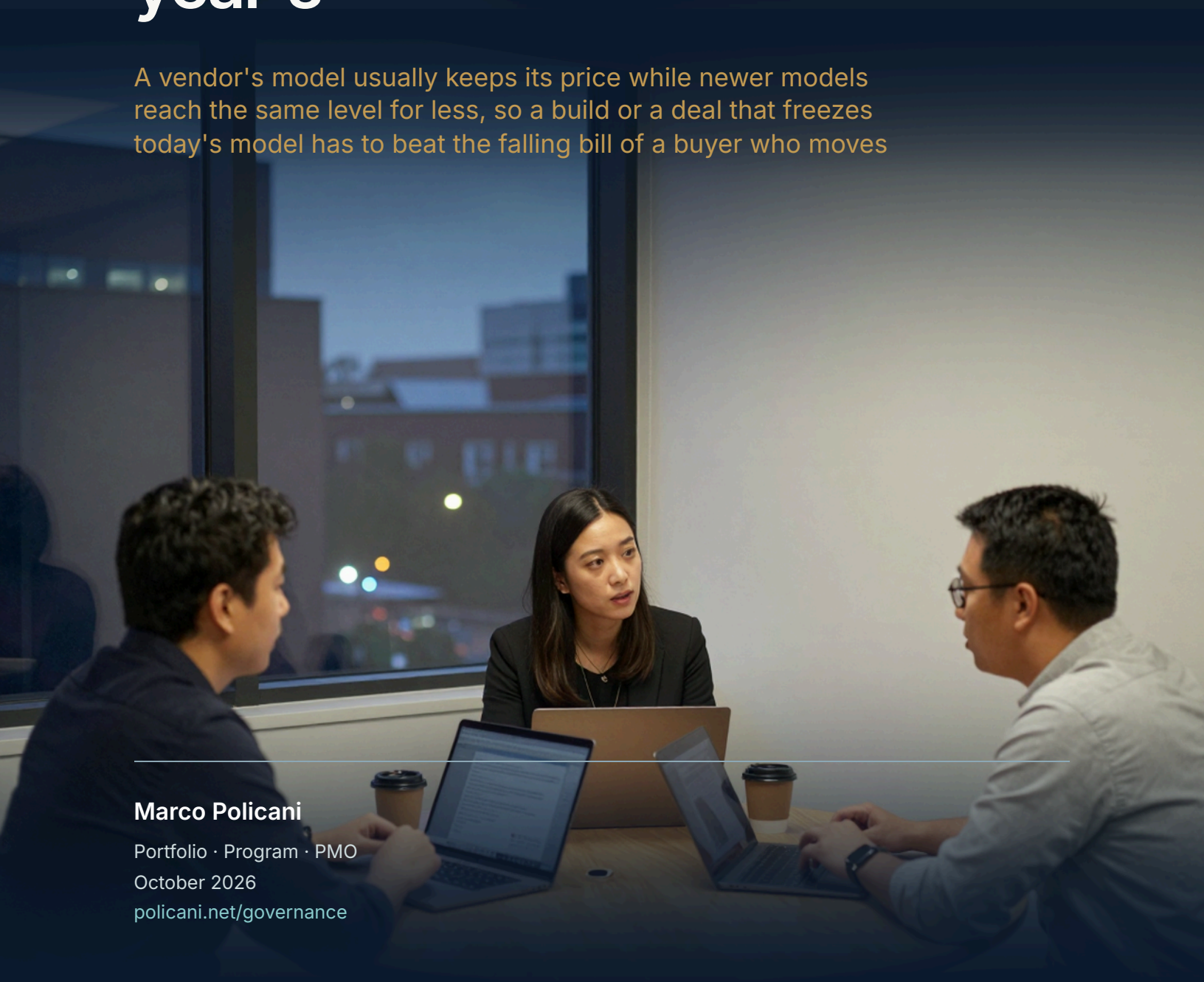
WHITE PAPER · PORTFOLIO AND AI GOVERNANCE

An AI build that beats today's price may not beat next year's

A vendor's model usually keeps its price while newer models reach the same level for less, so a build or a deal that freezes today's model has to beat the falling bill of a buyer who moves



Marco Policani

Portfolio · Program · PMO

October 2026

policani.net/governance

THE CENTRAL ARGUMENT

A business case for an AI build or a long model deal usually shows its saving against what a vendor charges today. A vendor's model generally keeps its launch price; the price of doing a task at a set level falls because newer models reach that level for less. So today's price held flat assumes the business never moves to a cheaper model. A build whose running cost is committed, or a deal that pins a model, should be judged against the bill of a buyer who moves at a realistic pace, and the case should show how fast prices would have to fall to wipe out its saving.

A team asks you to fund an AI build that cuts running cost. They might want to run an open-weight model, one whose maker lets anyone download and run it, on servers you own. Or they might tune a smaller model for one task by training it further on your own examples. Or a vendor offers a discount if you sign a multi-year deal that fixes one model and its price per token, the small piece of text vendors charge by. Either way, the business case shows the saving against what you pay a vendor today, times the use you expect. You have heard that AI keeps getting cheaper. You also see your own AI bill rising, and you have no clear way to square the two.

If the case holds today's price flat, it rests on a forecast it does not state: that you will never move to a cheaper model. If it assumes a gentle fall, it rests on a rate it has not checked against the falls recorded. A vendor's model generally keeps the price it launched at. What gets cheaper is reaching the same level of performance, because newer models reach that level for less. A buyer who moves to the cheapest model that does the task well enough gets the lower price. A build whose running cost you keep paying, or a deal that pins one model, does not.

Take a build that halves today's running cost for three years. Suppose the price of doing the task well enough fell about 1.7 times a year, so that each year it cost about 60 percent of the year before. If you moved, at no cost, as soon as a cheaper model appeared, the build's whole saving would be gone by the end. If you moved once a year, it would take a fall of about 2.7 times a year. In the main study cited below, on the one test where prices were tracked by level, prices fell faster than that at the middle and top levels, and about 1.7 times a year at the lowest.

So make every AI cost case say which price it is buying: the price of doing the task at a set level, or the price of always running on the best model. Show the yearly fall in price that would wipe out the saving, at the pace you would really move, with the cost of each move counted. Set it beside the falls recorded for a level like the one your task needs. For a deal, name what it fixes and for how long. For a need that must keep up with the best model, plan for a rising bill. For a build meant to beat the general models, name the task and the lead it must hold, and check that lead at each review against the general model on sale at that time.

ARGUMENT AT A GLANCE

What an AI cost case should compare

What the case assumes

Today's vendor price, held flat, which assumes you never move.

What the market does

Newer models reach the same level for less; the fall comes to buyers who move.

What to show

The yearly fall that would erase the saving, at your real pace of moving.

Applies to the running cost of AI models priced by use, and the builds and deals justified against it.

Two prices, moving in opposite directions

"The price of AI" is really two prices. Hans Gundlach and colleagues at MIT tracked what it cost to run AI models on public tests from April 2024 to November 2025, using the cheapest price any provider listed [1]. For models that offered the best performance for their price, the cost of reaching a fixed score fell about 5 to 10 times a year. Over the same period, the cost of running the best-scoring model on a test rose about 3 to 18 times a year, "due to bigger models and larger reasoning demands" [1]. In their words, "a fixed capability level becomes dramatically cheaper over time," while "achieving frontier performance continues to demand higher frontier expenditure" [1].

That is how AI can get cheaper while your AI bill rises. A second study points the same way. Mert Demirer, Andrey Fradkin and colleagues studied usage on two large marketplaces where businesses buy AI models [2]. The price of each level of performance fell steeply, yet the average price paid per token stayed roughly flat, which they read as "consistent with demand for superior intelligence" [2]. Buyers took the gains as better quality, not as lower bills.

A bill can rise for several reasons: more use, more words processed per task and moving up to better models. Only the last is a choice about what the task needs. So it helps to sort each need into one of two kinds. A fixed level means the task done as well as it is done now. Once you set it, even "as good as today's best model" is a fixed level. Keeping pace means the task must always run on whatever model is best. A fixed level sits on the falling price. Keeping pace sits on the rising one. The tests in these studies stand in for real business tasks, and that limit matters.

Why the price you pay stays put

If the price of a fixed level is falling so fast, why doesn't your bill for the same task fall with it? Because the fall usually does not come to the model you already use. Demirer and colleagues found that "closed-source models exhibit stable pricing, with most variants maintaining their launch price throughout their lifecycle" [2]. A closed model, one whose maker keeps it private and sets its price, changed price in 1.8 percent of weeks. New versions "often enter at prices comparable to those of their predecessors" [2]. Gundlach and colleagues noted the same pattern in their data, and found that some older models cost more when cheap providers stopped offering them [1].

Open-weight models are different. Many companies can host them, and those hosts compete. Their prices changed more than twice as often as closed models' and tended to fall [2]. So if you buy an open-weight model from a host, part of the fall comes to you without moving.

For a vendor's closed model, the fall reaches you only if you move to a newer, cheaper model that does the task well enough. Moving costs something. Someone has to test the task again, change the instructions the model is given and adjust the systems around it. And businesses do not always move for price. Demirer and colleagues found that when Anthropic released new models in its Sonnet line, buyers on one marketplace moved to them quickly. But what they moved to was a better model at about the same price, from the same vendor. Other new models did not draw buyers away so quickly. And on one large cloud, most firms put more than 90 percent of their use on a single model [2]. How often your business would really move is a fact to find out, not to assume.

So a build or a deal has to beat more than "keep paying today's price." It has to beat the cheaper of two things: staying where you are, or moving to the cheapest model that does the task well enough, at the pace you would really move, and paying for each move.

How fast the price of a level falls

The fall is real, but it varies a great deal. Gundlach and colleagues measured it on three tests and gave each estimate a range of uncertainty [1]. On GPQA-Diamond, a set of hard science questions, the best-value models got about 5.3 times cheaper a year (somewhere between 2.4 and 11.5). Counting every model, not only the best value, it was about 3.8 times (2.1 to 6.8). For open-weight models alone, counting every one, it was about 1.2 times (0.61 to 2.4). On a math test, AIME, the best-value models fell about 11.7 times a year and all models about 7.0. On a coding test, SWE-bench Verified, the data were so thin that the range for the best-value models runs from 0.68 to 32. A figure below 1 would mean prices rose, so that test cannot rule out a flat price. Counting all models rather than the best-value ones made the fall "smaller by almost a factor of 2" [1].

The fall also depends on the level. Only GPQA-Diamond was split by level. Among the cheapest models scoring 20 to 40 percent on it, the price fell about 1.7 times a year (1.4 to 2.2). Among open-weight models scoring 40 to 60 percent, it fell about 7.2 times (4.5 to 11.7); the authors say their estimate for all models at that level does not match the data well. Among models scoring 60 to 80 percent, it fell about 31 to 32 times a year, with very wide ranges [1]. In plain terms, a fall of 1.7 times a year means that a year later the same work costs about 60 percent of what it did. A fall of 30 times means it costs about 3 percent.

Three cautions apply. These are list prices from the cheapest provider. A business that may use only approved providers, or one vendor, may well see a slower fall, perhaps nearer the all-model rates; that is this paper's judgment, not the study's finding. The figures cover about 19 months, in a study not yet reviewed by other researchers. And test scores are not business tasks. Deciding where your task sits is a judgment you make. One practical way: find the cheapest model on sale today that does the task well enough, and see how far below the best it sits.

Still, every central estimate the study gives for the price of a fixed level is a fall, from about 1.2 to about 32 times a year. On GPQA-Diamond and AIME, a flat price lies outside the range of uncertainty for the best-value models and for all models taken together; only the series for open-weight models alone on GPQA-Diamond cannot rule it out. So holding today's price flat is not a neutral choice. It is the forecast for a business that never moves.

What that does to a build's saving

Here is the arithmetic. It is this paper's own, with numbers chosen to show the method. They are illustrations, not findings.

The argument covers a build whose running cost you keep paying even after the market price drops below it. That is true of servers you own, of computing capacity reserved and paid for over a fixed term, or of a model made by tuning one particular model and never tuned again. A build you could switch off at no further cost is a different case: you would keep what it saved and move on.

Say the build halves today's running cost for three years, paid as it runs, with nothing spent up front. Against today's price held flat, it saves a year and a half of today's bill for that task.

Now say the cheapest model that does the task well enough gets 1.7 times cheaper each year, and you would move to each cheaper model as soon as it appeared. The market price drops to the build's cost in about 16 months. Until then the build banks a saving. After that, the buyer who moves pays less than the build every month. By the end of three years the two come out even. The whole saving is gone.

The pace of moving matters a great deal. If you would move only once a year, prices would have to fall about 2.7 times a year to wipe out the saving. If each move also cost a tenth of today's yearly bill, and you moved twice in three years, prices would have to fall about 4.1 times a year. If you would never move, the build keeps its whole saving. At a gentler fall of 1.26 times a year, moving as soon as a cheaper model appeared, the build keeps about 44 percent of the saving its case claimed. Money spent up front on the build has the opposite effect to the cost of moving: it lowers the fall needed to wipe out the saving.

Set those rates beside what was recorded. At the lowest level measured, prices fell about 1.7 times a year. There, a build keeps all of its saving only if you never move. It keeps about 30 percent if you move once a year, about 40 percent if each move also costs a tenth of a year's bill, and none if you move at no cost as soon as a cheaper model appears. At the middle and top levels, the central estimates were well above all three rates, so a build there can lose its saving entirely.

The rates counting every model are mixed. On AIME, 7.0 a year, the saving is gone even after moving costs. On GPQA-Diamond, 3.8, almost nothing is left. On the coding test, 1.6, and for open-weight models on GPQA-Diamond, 1.2, a build keeps roughly half its saving or more, though neither range rules out no fall at all. These results assume yearly moves that each cost a tenth of a year's bill.

None of this makes builds a mistake. But a case cannot tell you whether a build is a mistake until it shows the fall that would wipe out the saving, at your pace of moving, beside the falls recorded at a level like yours. A build that runs open-weight models and keeps moving to newer ones gets some of the fall itself. The best-value open-weight models on GPQA-Diamond fell about 4.6 times a year [1]. But such a build also pays its own cost of moving, and counting every open-weight model on GPQA-Diamond, the fall was only about 1.2 times a year.

What a long deal fixes

The same reasoning applies to contracts. A deal that pins one model at a price per token behaves like a build whose cost you keep paying. Whether it gives up the falling price depends on what exactly it pins. How much it gives up depends on how long the deal runs. In return, it protects you if an older model costs more or is retired. Older models have cost more when cheap providers stopped offering them [1], and some have been retired [2]. Weigh that protection against the fall you give up. A pinned deal can also carry a clause that cuts its price when the vendor's newer models reach the same level; that keeps part of the fall, if the vendor will agree to it.

Some deals work differently. A commitment that fixes how much you will spend, but lets you move to newer models, keeps the falling price per task. Its risk is the other way: if your use does not grow, the money you committed can outrun what you need. Whether such terms are on offer depends on the vendor. They come from contract practice, and the studies cited here do not cover them. The familiar advice to keep contracts short points in the right direction, but the question to ask is sharper: what does this deal fix, and for how long?

When the build is meant to lead

Some builds aim to beat the general models, not to cost less. For these, the model they must beat keeps getting better, and reaching its level keeps getting cheaper.

Bloomberg trained its own finance model with 50 billion parameters, the adjustable values that set a model's size. It used Bloomberg's financial archives plus a roughly equal amount of public text. Training ran on 512 GPUs, the specialized chips used to train AI models, for about 53 days [3]. Bloomberg described the model on 30 March 2023 and compared it with the three closest open-weight models. OpenAI had described GPT-4 fifteen days earlier [4]. Researchers at Queen's University and J.P. Morgan then ran GPT-4 on the five public finance tests where Bloomberg had reported scores. GPT-4 scored higher on four of them. Bloomberg's model scored higher than GPT-4 at picking out names such as companies and people [5]. The authors suggest GPT-4's larger size as the reason it scored higher. Bloomberg's internal tasks were not compared, and nothing here says the build failed.

The same study also found where small models win. Small models trained for one narrow task beat GPT-4. On finding the relationship between two named things in a sentence, a tuned small model scored 56.30 to GPT-4's 46.87. On picking out names, a model trained on similar financial text scored 82.70 to 56.71 [5]. The authors conclude that for structured tasks like these, "the pretraining plus fine-tuning paradigm is still a leading option" [5].

So a lead is real only against the general model on sale at the time, at that model's price. Over the period measured, once general models had reached a level, the price of reaching it fell, at rates that varied by level [1]. A case for a leading build should name the task and the size of the lead it must hold. It should expect the general model it must beat to change during the build, and check the lead at each review against the general model on sale at that time. In that study, narrow, structured tasks were where small tuned models held a lead.

The objections

"Those falls were a burst from a new technology. The careful figure is today's price." The case does not need to bet on a rate. It shows the fall that would wipe out the saving, beside the falls recorded, and it shows the result if you never move. Today's price held flat is the forecast for a business that never moves, and only for one that never moves. The study's window also starts after the very first stage of the technology, which earlier and faster estimates included, though 19 months is still a short record.

"We cannot use the cheapest provider." Then use the providers you can use. The fall will be slower, and the comparison still applies.

"Moving models is costly and risky, and firms mostly stay on one model." Agreed. The cost of moving and how often you would really move belong in the comparison. Where they are large, the build or the pinned deal may win, and the comparison will show it.

"We run our own model to control our data, not to save money." Then the case rests on data control and should say so. This paper is about the cost argument.

"A pinned price protects us from increases." It does, for an older model that might cost more or be retired. Weigh that against the fall it gives up.

What to put in the case

Each cost case for a build or a deal gets one more comparison. First, name which price it buys: a fixed level, or keeping pace with the best. For a fixed level, state the level the task needs, the providers you can actually use, how often you would really move and what each move costs. Then show the yearly fall that would wipe out the saving at three paces: never moving, moving once a year, and moving as soon as a cheaper model meets the need. Set those beside the falls recorded for a level like yours. For a deal, say what it fixes and for how long. For keeping pace, plan for a rising bill. For a build meant to lead, name the lead and check it at each review against the general model on sale at that time.

Adding this is one more comparison in a case your team already prepares, though it needs real inputs: a test of the cheapest model that does the task well enough, the providers you can use, what a move costs and how often you would really move. Whether that effort pays is a judgment, not a measured cost. It is most worth doing where the build or the deal is large and the task sits at a level where prices have fallen fast.

The argument has limits. It covers the running cost of AI models priced by use, and the builds and deals justified against that cost. It does not cover seat licenses, integration, data preparation, the cost of changing how people work, or staff costs, none of which is shown to fall this way. It does not cover whether to use AI at all, or builds justified mainly by control of data or speed. It does not claim prices will keep falling at any rate. It claims that a case judged against today's price held flat assumes you will never move, and that the case should show what happens if you would.

The question for the case changes from "how much does this save against today's price?" to "how fast would prices have to fall, at the pace we would really move, for the saving to disappear?"

Sources

1. Hans Gundlach, Jayson Lynch, Matthias Mertens and Neil Thompson, MIT FutureTech. The Price of Progress: Price Performance and the Future of AI (arXiv 2511.23455, 2026).
2. Mert Demirer, Andrey Fradkin, Nadav Tadelis and Sida Peng, NBER Working Paper 34608. The Emerging Market for Intelligence: Pricing, Supply, and Demand for LLMs (2025).
3. Shijie Wu, Ozan Irsoy, Steven Lu and others, Bloomberg. BloombergGPT: A Large Language Model for Finance (arXiv 2303.17564, 2023).
4. OpenAI. GPT-4 Technical Report (arXiv 2303.08774, 2023).
5. Xianzhi Li, Samuel Chan, Xiaodan Zhu, Yulong Pei, Zhiqiang Ma, Xiaomo Liu and Sameena Shah, EMNLP 2023 Industry Track. Are ChatGPT and GPT-4 General-Purpose Solvers for Financial Text Analytics? A Study on Several Typical Tasks (2023).